Grind Karo Coach Admin — Copy Email on Athlete Page

30-Minute Screen Recording Guide (PDF companion)

Task title: Copy email on athlete page
Application: Skanda Global Task recorder
Repository: `Grind-karo-Admin` (Vite + React 19 + TypeScript + Tailwind)
Estimated recording length: ~~30 minutes~~
~~**Coding on camera:** ~6–8 minutes~~ (25% of total)
Walkthrough + verification: ~22–24 minutes

Recording brief (portal / Sonar prompt)

Use this as the **Goal** line in Word or Google Docs for your intro (under 20 seconds on screen):

> *Add a **Copy email** button on the athlete page in the Grind Karo coach admin. When viewing `/users/:id`, the coach can copy the athlete's email in one click and see a short **Copied** confirmation. Copy the same email string already shown in the header. Do not change the user record or call update APIs.*

Full studio prompt (structure reference):

- ~5–7 min: project walkthrough (code structure, config, inputs, current behavior).
- ~20–30%: live coding from **TODO 1–6** in starter code.
- Remainder: run app, test scenarios, DevTools inspection, lint/build, closing summary.

Table of contents

1. What you are building
2. Prerequisites and inputs
3. Project walkthrough (5–7 minutes on camera)
4. Starter code and TODO map
5. Implementation — TODO-by-TODO
6. Execution and verification (extended)
7. Minute-by-minute recording plan
8. Closing summary script
9. Deliverables checklist

---

1. What you are building

Coaches open an athlete at `/users/:id`. The page header shows the athlete name, **email** (with mail icon), phone, role, and join date. Email comes from the existing user payload (`user.email`) loaded by `useUserDetail`.

Your change: Add a **Copy email** control beside the displayed email. One click copies **exactly that string**, shows **Copied** for two seconds, and does **not** call any update API.

Pattern: Small presentational component under `src/components/users/`, wired once in `UserDetailPage.tsx` header row — same placement style as optional **Copy phone** on the phone row.

---

2. Prerequisites and inputs

2.1 Input files (deliverables naming)

Deliverable name	What it is
input 1.rtf	Empty metadata template from the portal
input 2 (optional)	Task brief / Sonar Webinar prompt text
artifact 1.png	Screenshot of athlete header *before* Copy email (optional)
final output 1/	This repo after your recording (with TODOs completed)
final output 2.mp4	TaskRecorder screen capture
final output 3.rtf	Filled metadata (Topology, Request, Task Description)

2.2 Environment

1. Copy `.env.example` → `.env` (do **not** commit `.env`):

   ```
   VITE_API_BASE_URL=https://api.grindkaro.in
   ```

2. Install and run:

   ```
   npm install
   npm run dev
   ```

   Default Vite URL: **http://localhost:5173**

3. **Browser:** Google Chrome or **Brave** (match Metadata Topology). Allow clipboard when prompted.

4. **Editor:** Visual Studio Code with repo root `Grind-karo-Admin` open (built-in terminal for `npm run dev`).

2.3 Accounts and test data (PII hygiene)

Use **fake seeded users** only on camera (TaskRecorder): e.g. **Test Athlete**, `test1@example.com`, fake phone. Turn on **Do Not Disturb**; no real names, Gmail, or revenue in frame.

Prepare **two** athletes:

Athlete	Email	Use case
A	`coach-test-a@example.com` (example)	Happy path copy + paste
B	Long or plus-address email	Paste matches full displayed string

Note UUIDs from the URL: `/users/<uuid>`.

---

3. Project walkthrough (5–7 minutes on camera)

Move the mouse slowly; narrate **what**, **where**, and **why**. Pause ~1 second after each click.

3.1 High-level architecture (1 minute)

Narration:
"This is the Grind Karo **coach admin** — a Vite + React SPA. It calls the GrindKaro REST API through Axios. Auth is JWT in `localStorage` via Zustand. Pages cache server data with TanStack React Query."

Mouse actions:

- Open `package.json` → `"dev": "vite"`, `react-router-dom`, `@tanstack/react-query`, `axios`.
- Open `src/main.tsx` → `QueryClientProvider` wrapping `App`.

3.2 Routing and athlete page (1.5 minutes)

Narration:
"Routes are in `App.tsx`. Athlete detail is `UserDetailPage` on `/users/:id`, behind staff routes."

Mouse actions:

- `src/App.tsx` → find `path="/users/:id"` and `UserDetailPage`.
- Open `src/pages/UserDetailPage.tsx` → scroll to header (back arrow, name, email/phone row).

3.3 Configuration (45 seconds)

Narration:
"API base URL is `VITE_API_BASE_URL` from `.env`. Copy email does not add new env vars — we only read `user.email`."

Mouse actions:

- Open `.env.example`.
- Open `src/services/api.ts` → `import.meta.env.VITE_API_BASE_URL`.

3.4 Data flow for email (1.5 minutes)

Narration:
" `useUserDetail(id)` fetches the user. The header renders `user.email` next to the Mail icon. My button receives that same prop — no extra API."

Mouse actions:

- In `UserDetailPage.tsx`, find `const { user, ... } = useUserDetail(...)` (or equivalent destructure).
- Open `src/hooks/useUserDetail.ts` → show query key and user fetch (briefly).

3.5 Reference pattern — Copy phone (1 minute)

Narration:
"Copy phone is already implemented in `CopyPhoneButton` — same clipboard API, same two-second **Copied** flash, same ghost button styling. My task is to mirror that for email in `CopyEmailButton`."

Mouse actions:

- Open `src/components/users/CopyPhoneButton.tsx` — walk through constant, `useEffect` timer, `copyPhone`, and button labels (do **not** copy-paste on camera unless your brief allows).

- Open `src/components/ui/Button.tsx` → `variant` and `size` props.

3.6 Starter / incomplete code (30 seconds)

Narration:
"Starter has **TODO 1–4** in `CopyEmailButton.tsx` and **TODO 5–6** in `UserDetailPage`. I'll implement those on camera."

Mouse actions:

- Open `src/components/users/CopyEmailButton.tsx` — scroll TODO comments (`return null` for now).
- In `UserDetailPage.tsx`, show TODO 5 import comment and TODO 6 JSX comment by the Mail row.

---

4. Starter code and TODO map

TODO	File	Approx. location	Purpose
TODO 1	`src/components/users/CopyEmailButton.tsx`	Top of file	Confirmation duration constant (2000 ms)
TODO 2	Same	Inside component	`copied` state + auto-reset `useEffect`
TODO 3	Same	Inside component	`copyEmail()` clipboard + error handling
TODO 4	Same	Return JSX	Ghost button: "Copy email" / "Copied"
TODO 5	`src/pages/UserDetailPage.tsx`	Import block ~line 66	Import `CopyEmailButton`
TODO 6	`src/pages/UserDetailPage.tsx`	Email header ~line 389–392	Render `<CopyEmailButton email={user.email} />`
(optional) **TODO 7**	`src/pages/UserDetailPage.tsx`	Phone header (else branch)	Copy phone task only — not part of Copy email DoD

Starter rules: TODO comments describe *what* and *why*, not full paste-ready code — type the snippets from §5 during the recording.

---

5. Implementation — TODO-by-TODO

> ***Recording tip:*** *Save after each TODO so HMR and the diff timeline show clear steps. Narrate file name and approximate line as you scroll.*

TODO 1 — Confirmation duration constant

File: `src/components/users/CopyEmailButton.tsx`
Locate: Below imports, before `CopyEmailButtonProps`.

Why: Coaches get a predictable two-second **Copied** flash before the label resets.

Narration while typing:
"I'm defining a single constant for the confirmation timeout so we can tune UX in one place."

Code to add:

```
const COPIED_CONFIRMATION_MS = 2000;
```

TODO 2 — Copied state and timer

File: `src/components/users/CopyEmailButton.tsx`
Locate: First lines inside `CopyEmailButton`, replace TODO 2 comment block.

Why: Toggle label Copy email → Copied → Copy email without reloading the page.

Narration:
"When `copied` is true, I schedule one timeout and clear it on unmount so we don't leak timers if the user navigates away."

Code to add:

```
const [copied, setCopied] = useState(false);

useEffect(() => {
  if (!copied) return;
  const timer = window.setTimeout(
    () => setCopied(false),
    COPIED_CONFIRMATION_MS,
  );
  return () => window.clearTimeout(timer);
}, [copied]);
```

TODO 3 — Clipboard handler

File: `src/components/users/CopyEmailButton.tsx`
Locate: After TODO 2 block, before return.

Why: `navigator.clipboard.writeText` is the standard secure clipboard API in modern browsers.

Narration:
"The `email` prop is exactly what the header displays — paste into Gmail or Slack should match character for character."

Code to add:

```
async function copyEmail() {
  try {
    await navigator.clipboard.writeText(email);
```

```
    setCopied(true);
  } catch {
    setCopied(false);
  }
}
```

TODO 4 — Button UI

File: `src/components/users/CopyEmailButton.tsx`
Locate: Replace `return null;` and TODO 4 comment.

Why: Visual consistency with other header ghost actions (muted text, compact height).

Narration:
" `type='button'` so we don't submit any form on the page. `void copyEmail()` because the handler is async."

Code to add:

```
return (
  <Button
    type="button"
    variant="ghost"
    size="sm"
    className="h-6 px-2 py-0 text-xs text-gray-600 dark:text-gray-300"
    onClick={() => void copyEmail()}
  >
    {copied ? "Copied" : "Copy email"}
  </Button>
);
```

Full component reference (after TODO 1–4):

```
import { useEffect, useState } from "react";
import { Button } from "@/components/ui/Button";

const COPIED_CONFIRMATION_MS = 2000;

type CopyEmailButtonProps = {
  email: string;
};

export function CopyEmailButton({ email }: CopyEmailButtonProps) {
  const [copied, setCopied] = useState(false);

  useEffect(() => {
    if (!copied) return;
    const timer = window.setTimeout(
      () => setCopied(false),
      COPIED_CONFIRMATION_MS,
    );
```

```
    return () => window.clearTimeout(timer);
  }, [copied]);

  async function copyEmail() {
    try {
      await navigator.clipboard.writeText(email);
      setCopied(true);
    } catch {
      setCopied(false);
    }
  }

  return (
    <Button
      type="button"
      variant="ghost"
      size="sm"
      className="h-6 px-2 py-0 text-xs text-gray-600 dark:text-gray-300"
      onClick={() => void copyEmail()}
    >
      {copied ? "Copied" : "Copy email"}
    </Button>
  );
}
```

TODO 5 — Import on athlete page

File: `src/pages/UserDetailPage.tsx`
Locate: Import section near other user components (~line 66).

Why: React needs `CopyEmailButton` in scope for the header JSX.

Narration:
"I'm importing next to the other user header widgets — `DeleteUserButton`, phone helpers, and so on."

Code: Uncomment / add (remove the `// TODO 5:` comment line):

```
import { CopyEmailButton } from "@/components/users/CopyEmailButton";
```

TODO 6 — Wire beside email in header

File: `src/pages/UserDetailPage.tsx`
Locate: Email `<span>` with `<Mail />` and `{user.email}`.

Why: Product requirement — control sits on the same row as the visible email.

Narration:
"I pass `user.email` as the prop so clipboard text always matches what we render."

Replace the TODO 6 comment with:

```
<CopyEmailButton email={user.email} />
```

---

6. Execution and verification (extended)

Spend most of the recording here: browser, paste tests, Network tab, lint/build.

6.1 Start the app

Run (VS Code integrated terminal):

```
npm run dev
```

Expected: `Local: http://localhost:5173/` . Log in as staff if prompted.

Narration:
"Vite HMR should show **Copy email** as soon as I save `UserDetailPage` after TODO 6."

6.2 Scenario 1 — Happy path

Steps:

1. **Users** → open **Athlete A** (`/users/<uuid-a>`).
2. Confirm **Copy email** appears beside the mail icon and address.
3. Click **Copy email** → label **Copied** (~2s) → **Copy email** again.
4. Click address bar or open Notes → **Cmd+V** (Mac) / **Ctrl+V** (Windows).

Expected paste: Exact `user.email` from the header.

Mouse: Highlight the email text, then the button — narrate they are the same string.

6.3 Scenario 2 — Second athlete / long email

Steps: Open **Athlete B** with a longer email.

Expected: Full string copies; button still resets after two seconds.

6.4 Scenario 3 — No API mutation

Steps:

1. DevTools → **Network** → **Fetch/XHR**.
2. Click **Copy email** several times.

Expected: No new `PUT` / `PATCH` / `POST` to user profile endpoints from this action.

Narration:
"Clipboard-only — network should stay quiet aside from existing page queries."

6.5 Scenario 4 — Double-click / rapid clicks

Steps: Click **Copy email** quickly three times.

Expected: Label may flicker **Copied**; no React errors in console; timer still resets to **Copy email**.

6.6 Scenario 5 — Navigation persistence

Steps: Copy email → navigate to another athlete → back.

Expected: Fresh **Copy email** label (not stuck on **Copied**).

6.7 Scenario 6 — Phone row unchanged (regression)

Steps: If **Copy phone** exists on the same page, click it once.

Expected: Phone copy still works; email button unaffected.

6.8 Scenario 7 — Lint and build

Run:

```
npm run lint
npm run build
```

Expected: Exit code 0; TypeScript resolves `CopyEmailButton` import.

6.9 Scenario 8 — Layout / theme (optional)

Steps: Narrow viewport; toggle dark mode if available.

Expected: Email row wraps; button remains tappable.

---

7. Minute-by-minute recording plan

Time	Focus	Mouse / keyboard	Narration hook	Expected on screen
0:00–0:20	Intro	Word/Docs Goal	One-line task; minimize	Goal visible briefly
0:20–2:00	Repo root	`package.json`, `src/`	Vite + React + Query	Tree structure
2:00–3:30	Routing	`App.tsx` → `UserDetailPage`	`/users/:id`	Route line
3:30–4:30	Config	`.env.example`, `api.ts`	`VITE_API_BASE_URL`	Env sample
4:30–5:30	Data	`useUserDetail`, header JSX	`user.email` source	Hook + render
5:30–6:30	Primitives	`Button.tsx`	Ghost/sm pattern	Component API
6:30–7:00	Starter	`CopyEmailButton` TODOs	Six TODOs total	Comments visible
7:00–8:00	**TODO 1–2**	Constant + state/effect	Timer cleanup	File saved
8:00–9:00	**TODO 3–4**	Handler + JSX	Clipboard labels	Component complete

9:00–10:00	**TODO 5–6**	Import + header wire	Prop = displayed email	Button in header
10:00–11:30	Dev server	`npm run dev`, login	Local admin	Athlete page
11:30–14:00	Scenario 1–2	Copy + paste	Happy path	Paste matches
14:00–15:30	Scenario 3	Network tab	No mutations	Quiet network
15:30–17:00	Scenarios 4–5	Rapid click + navigate	State resets	Stable UI
17:00–18:30	Scenarios 6–7	Phone regression + lint/build	Quality gate	Green commands
18:30–20:00	Scenario 8	Resize / theme	Layout	Wrapped header
20:00–22:00	Code review	Scroll final files	Two files touched	Clean diff
22:00–28:00	Buffer	Re-run Scenario 1 slowly	Reinforce DoD	Same expectations
28:00–30:00	Epilogue	Speak to camera	§8 script	Summary

Keep **typing ≤ ~10 minutes**; use buffer for verification if you type slowly.

---

8. Closing summary script

Narration (45–60 seconds, plus 3–4 min epilogue if portal requires):

"Today I added **Copy email** to the Grind Karo coach admin athlete header. `CopyEmailButton` writes `user.email` to the clipboard with the browser Clipboard API, shows **Copied** for two seconds, and clears the label with a `useEffect` timeout. On `UserDetailPage`, the button sits beside the mail icon and uses the same string we display. I verified paste content, rapid clicks, navigation, no extra API traffic on copy, and ran lint and build. Production touch points are `CopyEmailButton.tsx` and `UserDetailPage.tsx` only."

---

9. Deliverables checklist

- ☐ TODO 1–6 completed; `npm run dev` runs
- ☐ Scenarios 1–3 demonstrated on camera (paste + Network)
- ☐ `npm run lint` and `npm run build` shown (or run before record)
- ☐ **final output 2.mp4** — continuous take; coding visible in timeline
- ☐ **final output 3.rtf** — Topology chronological; Request matches Copy email brief
- ☐ **Deliverables.zip** — `input N`, `artifact N`, `final output 1` code only; no `.env`, `node_modules`, video, or Metadata inside zip

- [ ] **No PII** on screen — fake users only

Metadata quick reference (final output 3.rtf)

Topology (example):
`Brave (LOOKUP) → Brave (LOOKUP) → Visual Studio Code (LOOKUP) → Visual Studio Code (CREATE) → Visual Studio Code (MODIFY) → Brave (LOOKUP) → Terminal (LOOKUP)`

Use **Visual Studio Code** for the integrated terminal (not a separate Terminal app entry unless required).

Used applications: Brave or Google Chrome, Visual Studio Code

Request (short): Add Copy email on `/users/:id`; copy displayed `user.email`; show Copied confirmation; no user updates.

Task Description (numbered list example):

1. Open coach admin, navigate to `/users/:id`, point at email in the header.
2. In VS Code, implement `CopyEmailButton` (clipboard + Copied state) and wire it in `UserDetailPage`.
3. In Brave, click Copy email, confirm Copied and paste matches; confirm no profile API calls; run lint/build.

Document version: 2026-10-22 — Grind-karo-Admin Copy email recording guide.